

计算机地图制图

## 第四章 区域的生成算法



余接情

Email: [yujieqing@cumt.edu.cn](mailto:yujieqing@cumt.edu.cn)

## 章节安排

第一节 区域的有关概念

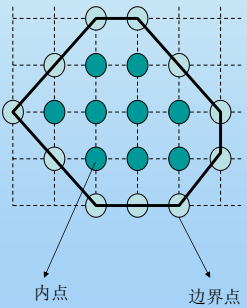
第二节 扫描转换算法

第三节 区域填充算法

## 第一节 区域的有关概念

### 1 区域的表示

顶点表示和点阵表示。

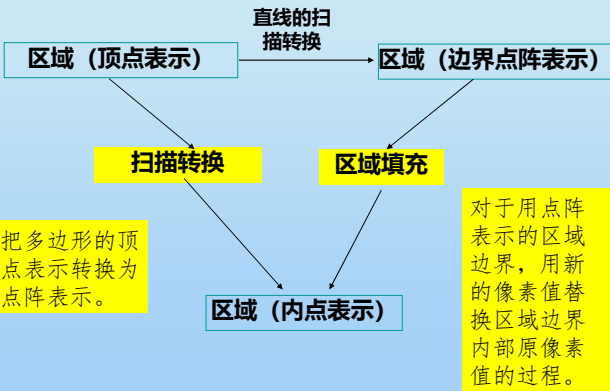


顶点表示→点阵表示：通过扫描转换实现。用于图形显示、矢量数据栅格化。

点阵表示→顶点表示：通过边界提取实现（模式识别），用于图形表示、建模，栅格数据矢量化。

## 第一节 区域的有关概念

### 2 区域的生成



区域 (顶点表示) → 区域 (边界点阵表示) (直线的扫描转换)

区域 (边界点阵表示) → 区域 (内点表示) (扫描转换)

区域 (边界点阵表示) → 区域 (内点表示) (区域填充)

把多边形的顶点表示转换为点阵表示。

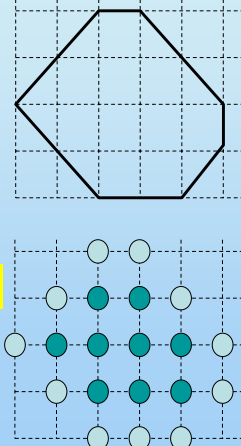
对于用点阵表示的区域边界，用新的像素值替换区域边界内部原像素值的过程。

## 第二节 多边形扫描转换算法

### 1、多边形扫描转换

扫描转换算法

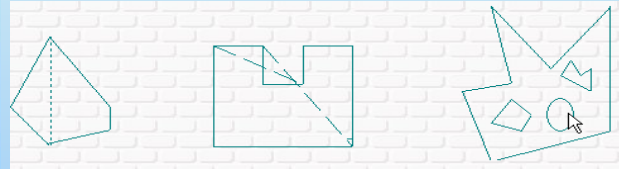
- 1 扫描线算法
- 2 边填充算法
- 3 边界标志算法



## 第二节 多边形扫描转换算法

### 2、扫描线算法

多边形分为三种：凸多边形、凹多边形、含内环的多边形。



凸多边形      凹多边形      环状多边形

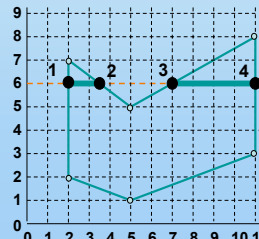
扫描线生成算法对多边形的形状没有限制，但多边形的边界不自交。

## 第二节 多边形扫描转换算法

### 2、扫描线算法

#### ● 总体思路（非自交多边形）

- (1) 求交点：计算该扫描线与多边形各边的交点；
- (2) 排序：将求出的交点按  $x$  坐标值排序；
- (3) 交点配对：1与2，3与4，...，每对表示一个区间；
- (4) 设置颜色：把区间内的像素置成前景颜色，区间外的像素置成背景色。

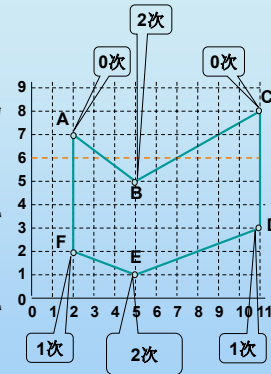


## 第二节 多边形扫描转换算法

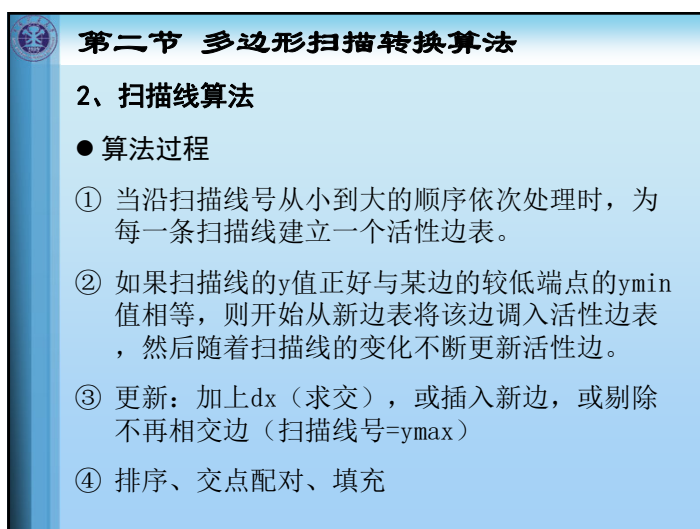
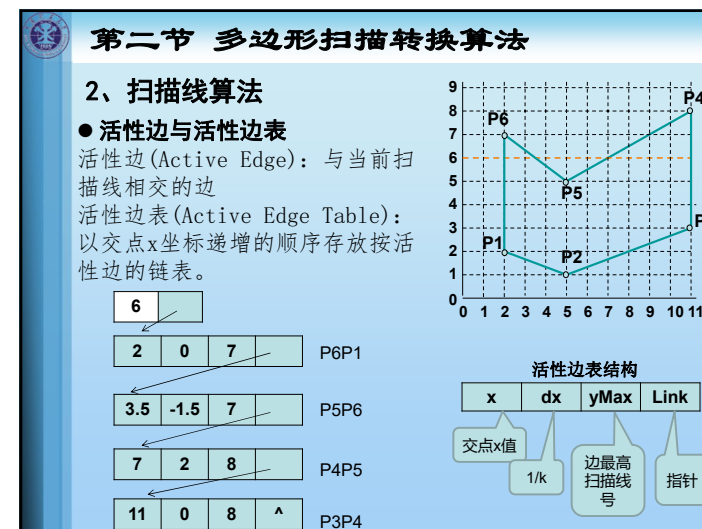
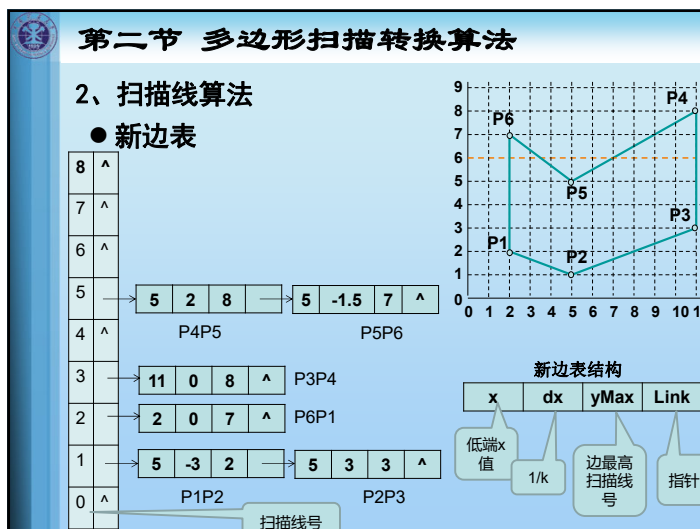
### 2、扫描线算法

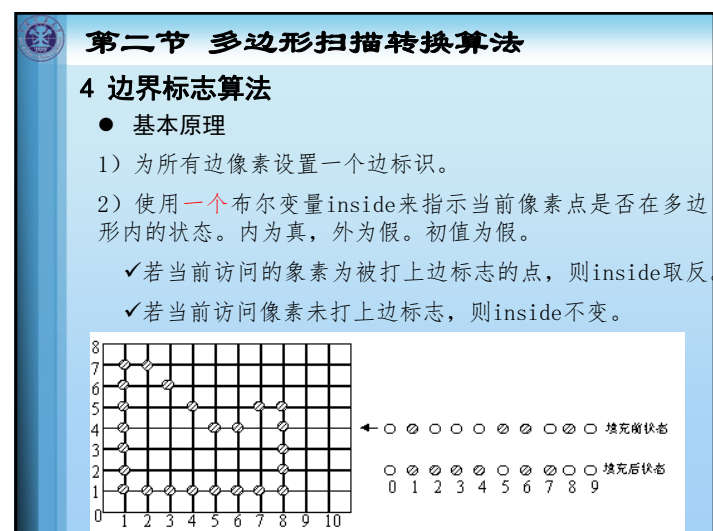
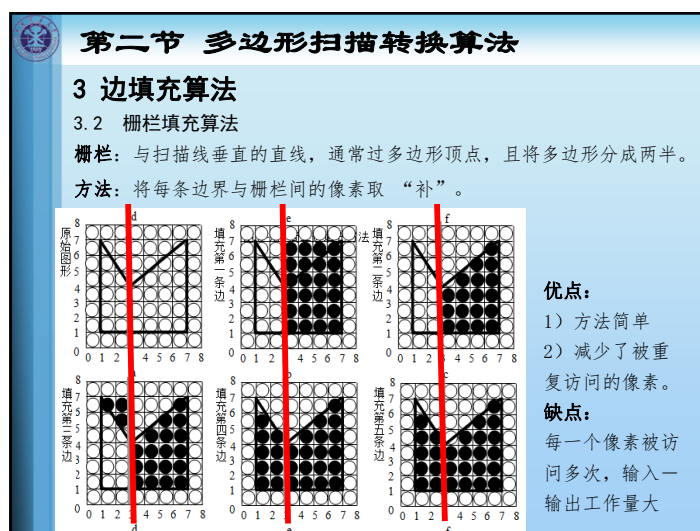
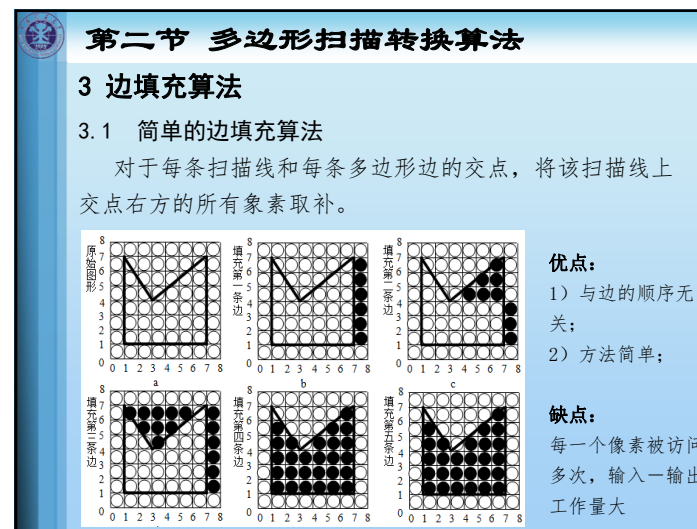
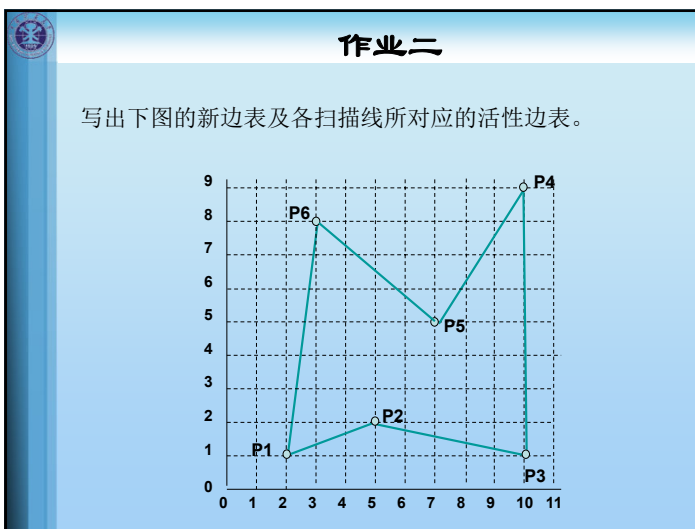
#### ● 顶点交点的取舍

- 1) 共享顶点的两条边分别落于扫描线的两侧，这时交点算一个，如F、D。
- 2) 共享顶点的两条边位于扫描线同一侧，且该顶点为局部最低点，则算2个，如B、E。
- 3) 共享顶点的两条边位于扫描线同一侧，且该顶点为局部最高点，则算0个，如A、C。



具体实现时，则检查顶点与相连两条边的另外两个端点的Y值的关系来判断。





## 第二节 多边形扫描转换算法

### 4 边界标志算法

✓ 优点

- 1) 不必建立、维护边表以及对它进行排序, 所以边标志算法比扫描线算法更适合于硬件实现, 执行速度比扫描线算法快到一至两个数量级。
- 2) 与边填充算法比, 不存在像素被访问多次的问题

✓ 缺点

时刻读取像素标志, 也较耗时  
部分边界像素被丢失

## 第三节 区域填充算法

### 1、区域填充

通常假定区域边界内部至少有一个像素是已知的, 称为种子点。

区域填充算法

- 简单种子填充算法
- 扫描线种子填充算法

## 第三节 区域填充算法

### 2、区域连通性

**四连通**: 从区域内一点出发, 通过上、下、左、右四个方向的移动, 在不越出区域的前提下, 能到达区域内的任意像素

**八连通**: 从区域内一点出发, 通过上、下、左、右、左上、右上、左下、右下8个方向的移动, 在不越出区域的前提下, 能到达区域内的任意像素

四连通区域边界 八连通区域边界

四连通 八连通

○ 边界点 ● 内点 ◐ 背景点

a、四连通区域和八连通区域的定义

## 第三节 区域填充算法

### 3、区域内外测试

**射线法**: 被测点P向 $y=-\infty$ 方向作射线, 奇数交点为内部, 否则为外部。

特殊情况处理

- 1) 若边在射线左侧, 计数
- 2) 若边在射线右侧, 交点无效, 不计数



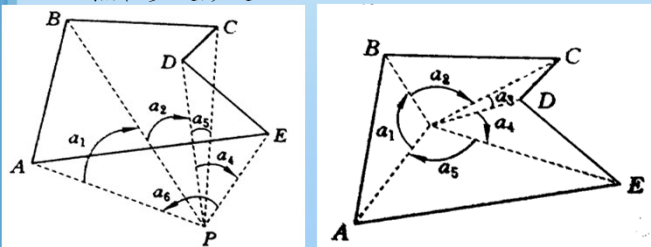
### 第三节 区域填充算法

#### 3、区域内外测试

**弧长法：**假设有向多边形的右侧为多边形的内部。以被测点为圆心做单位圆，将全部有向边向单位圆作径向投影，并计算其在单元圆上弧长的代数和。

0：点 在多边形外部； $2\pi$ ：点 在多边形内部；

$\pi$ ：点 在多边形边上



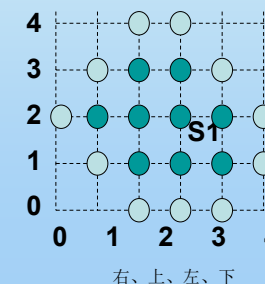
### 第三节 区域填充算法

#### 4 简单种子填充算法

##### • 基本原理

- 1) 若种子点不是边界也未填充，则填充并产生新的种子点；
- 2) 若种子点均已被填充，则函数结束。

```
SeedFill(x, y) {
  IF (x, y) 为边界点或已填充过, 则结束
  Else 填充当前颜色;
  SeedFill(x+1, y);
  SeedFill(x, y+1);
  SeedFill(x-1, y);
  SeedFill(x, y-1);
}
```



### 第三节 区域填充算法

#### 4 简单种子填充算法

##### ● 优缺点

- ✓ 优点：思路简单、程序简洁；
- ✓ 缺点：由于调用了递归函数，系统开销大，许多像素被重复访问，效率不高。

##### ● 递归调用的消除：顺序执行算法。

设置一个足够大的堆栈空间（Stack）来存放未填充的种子点。

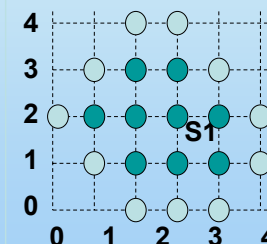


### 第三节 区域填充算法

#### 4 简单种子填充算法

##### ● 顺序执行算法

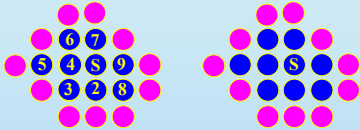
1. 初始化：种子像素入栈，当栈非空时，重复2~4的步骤；
2. 栈顶像素出栈；
3. 将出栈像素置为多边形颜色；
4. 按右、上、左、下顺序依次检查与出栈像素相邻的四个像素，若其中某个像素不在边界上且未置成多边形色，则该像素入栈；
5. 当堆栈为空时，算法终止。



### 第三节 区域填充算法

#### 4 简单种子填充算法

右、上、左、下



|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
|   |   |   |   | 5 |   |   |   |   |   |   |
|   |   | 3 | 4 | 6 | 6 | 7 |   |   |   |   |
|   | 2 | 8 | 8 | 8 | 8 | 8 | 8 | 9 |   |   |
|   | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |   |   |
|   | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 |   |
| S | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 | 9 |

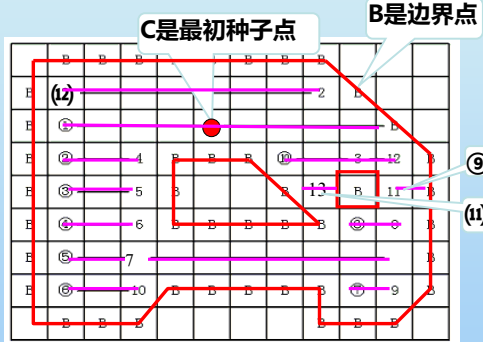
缺点：花费许多空间构造堆栈结构

### 第三节 区域填充算法

#### 5 扫描线种子填充算法

##### ● 基本原理

以种子所在扫描线进行从左到右填充直至边界为止，然后不断产生新的种子压入栈



压栈：先上后下，总是以最右边为种子点

① 或 (1) 表示出栈顺序，1表示压栈顺序

### 第三节 区域填充算法

#### 5 扫描线种子填充算法

##### ● 填充算法

- (1) 初始化。将算法设置的堆栈置为空。将给定的种子  $(x, y)$  压入堆栈。
- (2) 出栈。如果堆栈为空，算法结束。否则从包含种子像素的堆栈中取出栈顶元素  $(x, y)$  作为种子像素。
- (3) 区间填充。沿当前扫描线对种子像素的左右像素进行填充（像素值为 `new_color`），直至遇到边界像素为止，从而填满包含种子像素的区间。
- (4) 进栈。在  $x_l \leq x \leq x_r$  中，检查与当前扫描线相邻的上下两条扫描线是否全为边界像素(`boundary_color`)或者前面已经填充过的像素 (`new_color`)，是则转到步骤(2)，否则在  $x_l \leq x \leq x_r$  中把每一个区间的最右像素作为种子像素压入堆栈，再转到步骤 (2) 继续执行。

### 第三节 区域填充算法

#### 7 扫描转换与区域填充的比较

##### ● 联系

给定多边形的顶点，可用扫描转换算法生成得到多边形的区域；但若先用直线生成算法生成多边形的边，则该问题转换为区域填充问题，反之亦然。

##### ● 区别

- 1) 基本思想不同：前者将顶点表示转换为点阵表示，后者只改变了区域的填充颜色，未改变区域的表示方法。
- 2) 对边界要求不同：前者只要求扫描线与多边形交点个数为偶数，可以不封闭，后者要求是四连通或八连通封闭区域。
- 3) 基于的条件不同：前者从多边形的顶点出发，利用多边形的连贯性进行填充；后者要求给定种子点，根据连通性进行填充。





### 第三节 区域填充算法

#### 7 扫描转换与区域填充的比较

##### ● 联系

给定多边形的顶点，可用扫描转换算法生成得到多边形的区域；但若先用直线生成算法生成多边形的边，则该问题转换为区域填充问题，反之亦然。

##### ● 区别

- 1) **基本思想不同**：前者将顶点表示转换为点阵表示，后者只改变了区域的填充颜色，未改变区域的表示方法。
- 2) **对边界要求不同**：前者只要求扫描线与多边形交点个数为偶数，可以不封闭，后者要求是四连通或八连通封闭区域。
- 3) **基于的条件不同**：前者从多边形的顶点出发，利用多边形的连贯性进行填充；后者要求给定种子点，根据连通性进行填充。

